Assignment 1

Submitted By :-
Anish Mahajan (102303132)

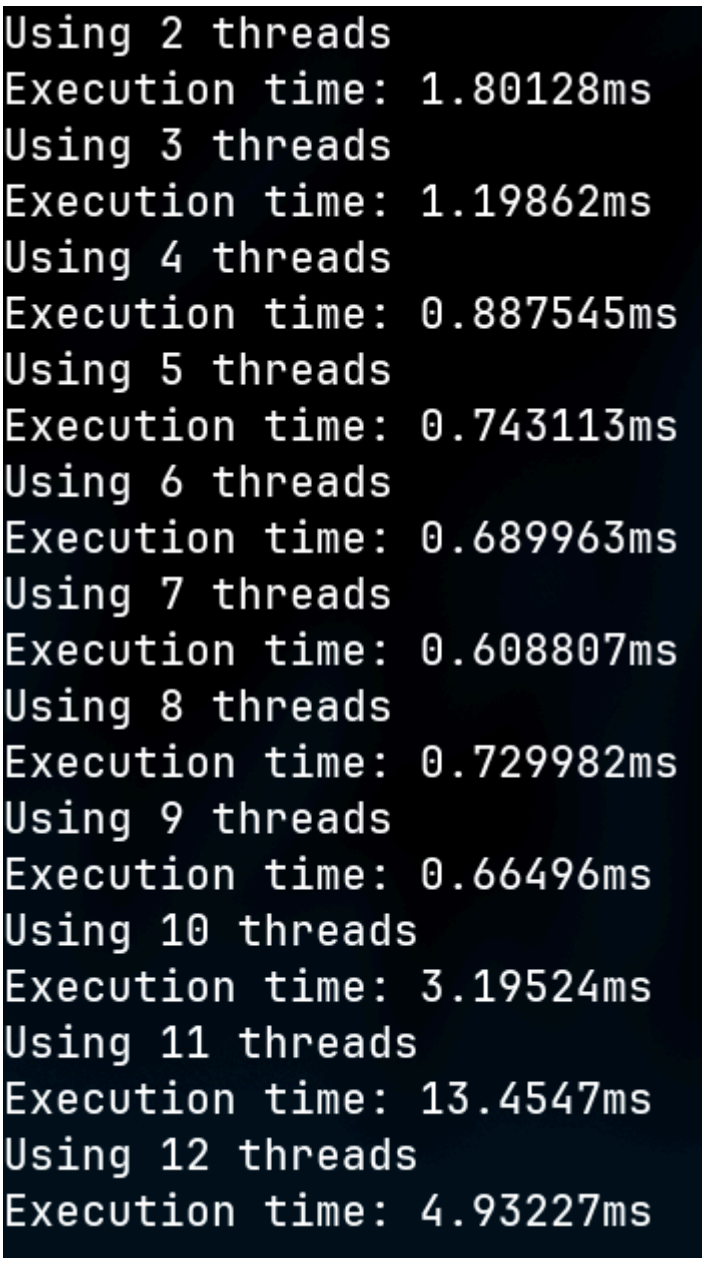
Q1. DAXPY Loop :-

Code :-

```
double get_random_number(){
	static thread_local std::mt19937 mt{std::random_device{}()};
	static thread_local std::uniform_real_distribution<double>
random_range{0.0,1.0};
	return random_range(mt);
}
int main(int argc, char** argv){
	if(argc < 2) [[unlikely]]{
		std::println("Usage: ./daxpy_loop scalar_value");
		return EXIT_FAILURE;
	}
	constexpr int SIZE{1<<16};
	int totol_num_threads{omp_get_max_threads()};
	double a{std::atof(argv[1])};
	std::vector<double> x(SIZE, 0.0);
	std::vector<double> y(SIZE, 0.0);

	for(int start_threads{2};start_threads <= totol_num_threads; ++start_threads){
		std::println("Using {} threads", start_threads);
		omp_set_num_threads(start_threads);
		auto start_time{std::chrono::steady_clock::now()};
		#pragma omp parallel for
		for(int i = 0;i<SIZE;++i){
			x[i] = get_random_number();
			y[i] = get_random_number();
			x[i] = a*x[i] + y[i];
		}
		auto end_time{std::chrono::steady_clock::now()};
		auto exec_time{end_time - start_time};
		std::chrono::duration<double, std::milli> ms{exec_time};
		std::println("Execution time: {}",ms);
	}
	return EXIT_SUCCESS;
```

}

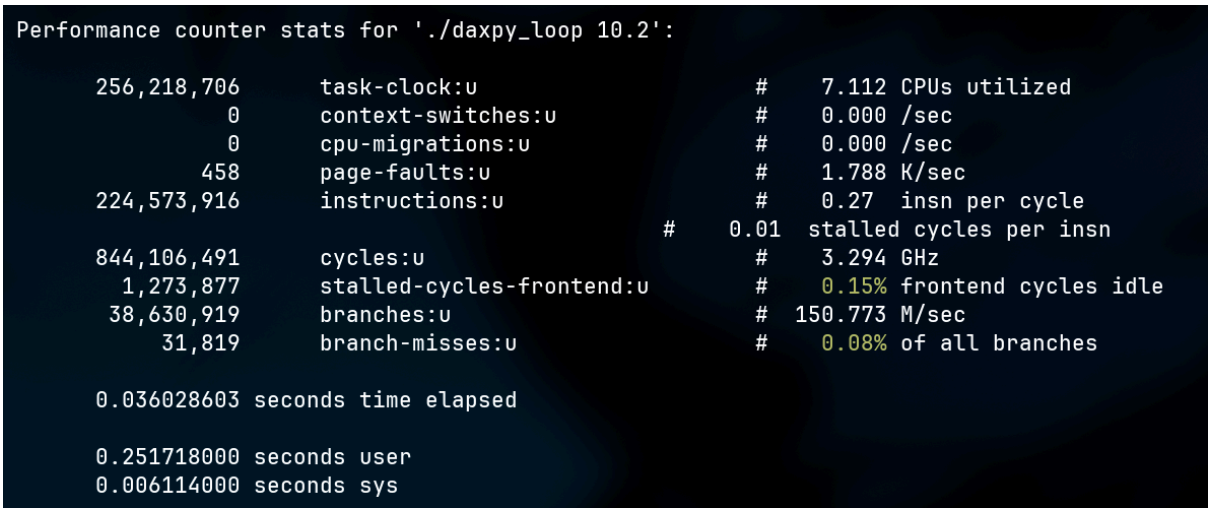
Program Output :-



Observation :-

After execution of the program we can see that after 7 threads the program performance deteriorates. This happens because my cpu has 6 cores, so when we create 7 or more threads, they cannot all run at the same time. The CPU wastes time constantly shuffling them around to give everyone a turn, and this shuffling takes longer than just letting 6 threads finish the job without interruption.

Observation from perf stat :-



```
Performance counter stats for './daxpy_loop 10.2':

      256,218,706      task-clock:u                     #    7.112 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
              458      page-faults:u                    #    1.788 K/sec
      224,573,916      instructions:u                   #    0.27  insn per cycle
                                                  #    0.01  stalled cycles per insn
      844,106,491      cycles:u                         #    3.294 GHz
        1,273,877      stalled-cycles-frontend:u        #    0.15% frontend cycles idle
       38,630,919      branches:u                       #  150.773 M/sec
           31,819      branch-misses:u                  #    0.08% of all branches

      0.036028603 seconds time elapsed

      0.251718000 seconds user
      0.006114000 seconds sys
```

Effective Parallelism: The metric 7.112 CPUs utilized confirms successful multithreading, with the program keeping ~7 cores active simultaneously. This aligns with the optimal execution times observed at 7-8 threads.

Memory-Bound Bottleneck: The low IPC (Instructions Per Cycle) of 0.27 indicates the CPU spends significant time waiting for memory access rather than computing.

Efficient Scheduling: The 0 count for context-switches and cpu-migrations during this specific run indicates stable thread scheduling without OS interference, which explains why performance was optimal before the thread count exceeded the hardware limits.

Q2. Matrix Multiplication :-

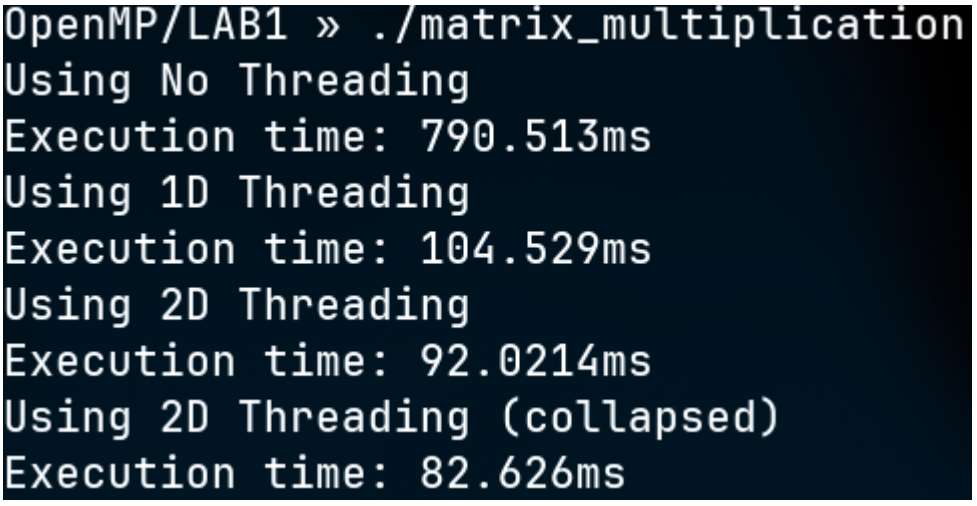
Code :-

```
double get_random_number(){
	static thread_local std::mt19937 mt{std::random_device{}()};
	static thread_local std::uniform_real_distribution<double>
random_range{0.0,1.0};
	return random_range(mt);
}
int main(){
	constexpr int ROW_SIZE{1000};
	constexpr int COL_SIZE{1000};
	std::vector<double> mat_a(ROW_SIZE*COL_SIZE, 0.0);
	std::vector<double> mat_b(ROW_SIZE*COL_SIZE, 0.0);
	std::vector<double> mat_b_T(ROW_SIZE * COL_SIZE, 0.0);
	std::vector<double> res(ROW_SIZE*COL_SIZE, 0.0);


	#pragma omp parallel for collapse(2)
	for(int i = 0;i<ROW_SIZE;++i){
		for(int j = 0;j<COL_SIZE;++j){
			mat_a[i*COL_SIZE+j] = get_random_number();
			mat_b[i*COL_SIZE+j] = get_random_number();
		}
	}


	#pragma omp parallel for collapse(2)
	for(int i = 0; i < ROW_SIZE; ++i){
		for(int j = 0; j < COL_SIZE; ++j){
			mat_b_T[j * ROW_SIZE + i] = mat_b[i * COL_SIZE + j];
		}
	}


	std::println("Using No Threading");
	auto start_time{std::chrono::steady_clock::now()};
	for(int i = 0;i<ROW_SIZE;++i){
		for(int j = 0;j<COL_SIZE;++j){
```

```
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum += mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
            }
            res[i*COL_SIZE+j] = sum;
        }
}
auto end_time{std::chrono::steady_clock::now()};
auto exec_time{end_time-start_time};
std::chrono::duration<double, std::milli> ms{exec_time};
std::println("Execution time: {}", ms);


std::println("Using 1D Threading");
start_time = std::chrono::steady_clock::now();
#pragma omp parallel for
for(int i = 0;i<ROW_SIZE;++i){
        for(int j = 0;j<COL_SIZE;++j){
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum += mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
            }
            res[i*COL_SIZE+j] = sum;
        }
}
end_time = std::chrono::steady_clock::now();
exec_time = end_time-start_time;
ms = exec_time;
std::println("Execution time: {}", ms);


std::println("Using 2D Threading");
start_time = std::chrono::steady_clock::now();
#pragma omp parallel for
for(int i = 0;i<ROW_SIZE;++i){
        #pragma omp parallel for
        for(int j = 0;j<COL_SIZE;++j){
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum += mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
            }
            res[i*COL_SIZE+j] = sum;
```

```
            }
        }
        end_time = std::chrono::steady_clock::now();
        exec_time = end_time-start_time;
        ms = exec_time;
        std::println("Execution time: {}", ms);


        std::println("Using 2D Threading (collapsed)");
        start_time = std::chrono::steady_clock::now();
        #pragma omp parallel for collapse(2)
        for(int i = 0;i<ROW_SIZE;++i){
            for(int j = 0;j<COL_SIZE;++j){
                double sum{0.0};
                for(int k = 0;k<ROW_SIZE;++k){
                    sum += mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
                }
                res[i*COL_SIZE+j] = sum;
            }
        }
        end_time = std::chrono::steady_clock::now();
        exec_time = end_time-start_time;
        ms = exec_time;
        std::println("Execution time: {}", ms);
        return 0;
}
```

Program Output :-



Observation :-

The implementation incorporates matrix transposition to enhance cache utilization and minimize memory latency. We also opted for a single flattened large vector rather than a vector of vectors. Since standard C++ vectors are heap-allocated, a nested vector structure results in non-contiguous memory blocks, which degrades cache performance and increases latency. By flattening the data structure, we ensured contiguous memory access. Furthermore, we observed that naive 2D threading was suboptimal, as OpenMP tends to discourage nested parallel regions, effectively reducing them to 1D threading. To mitigate this, we utilized the OpenMP collapse clause to merge the loop nests, thereby maximizing parallel efficiency.

Observations from perf stats :-

```
Performance counter stats for './matrix_multiplication':

    3,770,124,701      task-clock:u                     #    3.490 CPUs utilized
               0      context-switches:u               #    0.000 /sec
               0      cpu-migrations:u                 #    0.000 /sec
           1,898      page-faults:u                    #  503.432 /sec
   12,360,826,552      instructions:u                   #    0.89  insn per cycle
                                                  #    0.01  stalled cycles per insn
   13,934,518,485      cycles:u                         #    3.696 GHz
       80,740,635      stalled-cycles-frontend:u        #    0.58% frontend cycles idle
    1,056,305,600      branches:u                       #  280.178 M/sec
        4,068,987      branch-misses:u                  #    0.39% of all branches

      1.080113650 seconds time elapsed

      3.738261000 seconds user
      0.025863000 seconds sys
```

Parallel Utilization: The metric 3.490 CPUs utilized indicates that the program actively used approximately 3.5 logical cores on average.

Improved Computational Intensity: The IPC (Instructions Per Cycle) is 0.89. This reflects the computational nature of Matrix Multiplication (O(N^3)); the CPU performs more arithmetic operations for every piece of data fetched from memory, resulting in better pipeline utilization compared to memory-bound tasks.

Predictable Control Flow: The branch miss rate is exceptionally low at 0.39% (approx. 4 million misses out of 1 billion branches). This confirms that the standard triple-loop structure of matrix multiplication is highly predictable for the CPU's branch predictor, minimizing stalls caused by logical decisions.

Q3. Pi Calculation :-

Code :-

```
int main(){
	constexpr int num_steps{static_cast<int>(1e9)};
	double pi{};
	double sum{0.0};
	double step{1.0/num_steps};

	std::println("With No Threading");
	auto start_time{std::chrono::steady_clock::now()};
	for(int i = 0;i<num_steps;++i){
		double x = (i+0.5)*step;
		sum += 4.0/(1.0+x*x);
	}
	pi = step*sum;
	auto end_time{std::chrono::steady_clock::now()};
	auto exec_time{end_time-start_time};
	std::chrono::duration<double, std::milli> ms{exec_time};
	std::println("Execution Time: {}", ms);
	std::println("Pi (Serial): {}", pi);

	sum = 0.0;
	std::println("With Threading");
	start_time = std::chrono::steady_clock::now();
	#pragma omp parallel for reduction(+:sum)
	for(int i = 0;i<num_steps;++i){
		double x = (i+0.5)*step;
		sum += 4.0/(1.0+x*x);
	}
	pi = step*sum;
	end_time = std::chrono::steady_clock::now();
	exec_time = end_time-start_time;
	ms = exec_time;
	std::println("Execution Time: {}", ms);
	std::println("Pi (Parallel): {}", pi);
	return 0;
}
```

Program Output :-

```
With No Threading
Execution Time: 789.456ms
Pi (Serial): 3.1415926535899708
With Threading
Execution Time: 95.9613ms
Pi (Parallel): 3.1415926535898597
```

Observation :-

The parallel implementation of the Pi calculation demonstrated near-linear scalability, achieving an approximate 8.2x speedup by reducing execution time from 789.46ms (serial) to 95.96ms (parallel). This drastic improvement confirms that the workload is compute-bound, allowing the system to fully utilize the available processing cores without the memory bandwidth bottlenecks observed in previous array-based experiments. Additionally, the use of the **#pragma omp parallel for reduction(+:sum)** directive effectively managed data races without the overhead of explicit locks, though a minor discrepancy in the final Pi value was observed due to the non-associative nature of floating-point arithmetic when summing in parallel.

Observations from perf stat :-

```
Performance counter stats for './pi':

    1,855,427,504      task-clock:u                     #    2.087 CPUs utilized
                0      context-switches:u               #    0.000 /sec
                0      cpu-migrations:u                 #    0.000 /sec
              174      page-faults:u                    #   93.779 /sec
    7,274,692,773      instructions:u                   #    1.02  insn per cycle
                                                  #    0.00  stalled cycles per insn
    7,137,042,167      cycles:u                         #    3.847 GHz
        1,736,209      stalled-cycles-frontend:u        #    0.02% frontend cycles idle
      256,748,334      branches:u                       #  138.377 M/sec
           22,447      branch-misses:u                  #    0.01% of all branches

      0.888834015 seconds time elapsed

      1.850793000 seconds user
      0.006012000 seconds sys
```

Average Utilization: The metric 2.087 CPUs utilized reflects the program's hybrid structure. Since the execution includes a significant serial portion (approx. 789ms on 1 core) followed by a brief parallel portion (approx. 96ms on ~6 cores).
Compute-Bound Efficiency: The IPC (Instructions Per Cycle) is 1.02. This indicates the workload is Compute-Bound, heavily utilizing the CPU's arithmetic units for floating-point calculations rather than stalling on memory access.

Predictable Control Flow: The branch miss rate is negligible at 0.01% (approx. 22,000 misses out of 256 million branches). This confirms that the simple, fixed-iteration loops used for numerical integration are trivial for the CPU's branch predictor to anticipate, ensuring near-perfect pipeline efficiency.